

Guide for Contributors

This guide covers every Git command you'll need to contribute to **ui.venumity.com**. It explains **what** each command does, **when** to use it, and **why** it matters. You can keep this open while you work.

First-Time Setup

Run these commands only once when you start contributing.

1. Fork the repository (on GitHub)

Click the **Fork** button on the repo page. This creates your own copy under your GitHub account.

2. Clone your fork

```
git clone https://github.com/YOUR_USERNAME/ui.venumity.git
cd ui.venumity
```

When: right after forking.

Why: downloads your fork to your local machine so you can work on it.

3. Add the upstream remote

```
git remote add upstream https://github.com/thevinayakgore/ui.venumity.git
git remote -v
```

When: once, after cloning.

Why: links your local repo to the original repo so you can pull the latest updates.

4. Install dependencies

```
npm install
# or
pnpm install
```

When: after cloning, and any time `package.json` changes.

Why: installs all required packages so the project runs.

5. Start the dev server

```
npm run dev
```

When: every time you start working.

Why: launches the local development environment at `http://localhost:3000` to preview your changes.

Syncing Your Fork

Always sync before starting new work. This prevents merge conflicts and keeps your branch up-to-date.

```
git checkout main
git pull upstream main
```

```
git push origin main
```

Command	What it does	Why
git checkout main	Switch to your local main branch	You must be on main before syncing
git pull upstream main	Get latest changes from the original repo	Brings in new components, fixes, or doc updates
git push origin main	Update your fork on GitHub	Keeps your fork in sync for future PRs

When: before creating any new branch.

Why: starts your work from the latest code, avoiding outdated base.



Creating a Branch

Never work directly on `main`. Create a descriptive branch for each change.

```
git checkout -b <type>/<short-name>
```

Examples:

```
git checkout -b feat/spotlight-card      # new component
git checkout -b fix/button-hover-state   # bug fix
git checkout -b docs/update-readme       # documentation
git checkout -b a11y/modal-keyboard-nav  # accessibility
```

When: before making any code changes.

Why: isolates your work, makes PRs cleaner, and keeps main stable.

Branch naming rules:

- `feat/` – new component or feature
- `fix/` – bug fix
- `docs/` – documentation
- `style/` – visual/spacing changes
- `refactor/` – code improvement without behavior change
- `a11y/` – accessibility improvement



Making Changes and Committing

1. Check what changed

```
git status
```

When: after editing files, before staging.

Why: shows which files are modified, added, or deleted.

2. Stage your changes

```
git add .
```

or stage specific files:

```
git add components/venumity/cards/spotlight-cards/component.tsx registry/components.ts
```

When: after you're satisfied with your changes.

Why: moves files to the "staging area" — ready to be committed.

3. Commit with a clear message

```
git commit -m "feat: add spotlight card component"
```

Commit message format:

```
<type>: <short description>
```

Common types: `feat`, `fix`, `docs`, `style`, `refactor`, `ally`, `chore`.

When: after staging.

Why: saves a snapshot of your changes locally with a meaningful label.



Pushing and Creating a PR

1. Push your branch to your fork

```
git push origin feat/spotlight-card
```

When: after committing.

Why: uploads your branch to GitHub so you can open a Pull Request.

2. Open a Pull Request

1. Go to your fork on GitHub.
2. Click "**Compare & pull request**".
3. Fill in the PR template (title, description, screenshots).
4. Reference the related issue: `Closes #42`.
5. Click "**Create Pull Request**".

When: after pushing.

Why: asks the maintainer to review and merge your changes into the original repo.



Updating an Existing PR

If the maintainer requests changes, update your branch:

```
git checkout feat/spotlight-card    # switch to your branch
# make changes
git add .
git commit -m "fix: address review feedback"
git push origin feat/spotlight-card  # push again
```

When: after receiving review feedback.

Why: the PR updates automatically with your new commits.



Cleaning Up After Merge

Once your PR is merged, delete your local branch to stay organized.

```
git checkout main
git pull upstream main      # get the merged changes
git branch -D feat/spotlight-card  # delete local branch
```

Optional: delete the remote branch on your fork via GitHub UI or:

```
git push origin --delete feat/spotlight-card
```

When: after your PR is merged.

Why: keeps your local repo clean and prevents accidental work on stale branches.

Common Git Errors & Fixes

Error: `fatal: Need to specify how to reconcile divergent branches`

You pulled from `main` but local and remote histories have diverged.

Fix (merge – recommended):

```
git pull --no-rebase upstream main
```

Fix (rebase – cleaner history):

```
git pull --rebase upstream main
```

Error: `! [rejected] main -> main (non-fast-forward)`

Your fork's `main` is behind.

Fix:

```
git pull upstream main
git push origin main
```

Error: `Module not found: Can't resolve...`

A file import is broken.

Fix: locate the missing file, restore it or fix the import path, then:

```
git add .
git commit -m "fix: resolve missing module"
git push origin your-branch
```

Quick Reference Cheat Sheet

Command	When to use
<code>git status</code>	Check what has changed
<code>git checkout main</code>	Switch to main branch

<code>git pull upstream main</code>	Get latest from original repo
<code>git checkout -b feat/name</code>	Start a new feature branch
<code>git add .</code>	Stage all changes
<code>git commit -m "message"</code>	Save changes locally
<code>git push origin feat/name</code>	Upload branch to your fork
<code>git branch -D branch-name</code>	Delete local branch after merge
<code>git log --oneline</code>	View recent commits
<code>git diff</code>	See unstaged changes

✅ Workflow Summary

1. **Sync** → `git checkout main && git pull upstream main`
2. **Branch** → `git checkout -b feat/my-component`
3. **Code** → edit files, run `npm run dev` to test
4. **Stage** → `git add .`
5. **Commit** → `git commit -m "feat: add my component"`
6. **Push** → `git push origin feat/my-component`
7. **Open PR** on GitHub
8. **Update if needed** → commit and push again
9. **Clean up** after merge → delete branch

📋 Pro Tips — Do's and Don'ts

✅ DO

- Always pull the latest `main` before starting work
- Write clear commit messages and branch names
- Test your changes locally before pushing
- Follow existing code style and component patterns
- Update documentation when adding new components
- Delete local branches after merging

❌ DON'T

- Don't push directly to `main`
- Don't use `git push -f` unless absolutely necessary
- Don't commit large generated files or `node_modules`
- Don't mix unrelated changes in one PR
- Don't submit PRs without linking an issue (if applicable)

? FAQ

I'm new to open source — can I still contribute ?

Absolutely. Start with a `good first issue` or a documentation typo. We are happy to guide you.

Do I need to know TypeScript ?

Most components use TypeScript, but you can contribute simple docs or design changes without deep TS knowledge.

What if my PR is not perfect ?

We will leave friendly review feedback. PRs are a learning process no one expects perfection.

Can I propose a brand new component ?

Yes ! Open an issue with the `enhancement` label first, or start a discussion. Once approved, you can implement it.

How long does review take ?

Typically 1-3 days. Please be patient the maintainer is currently solo but actively engaged.



Thank You

⚡ Every contribution - no matter how small, helps make ui.venumity.com better. You are now part of the community. If you get stuck, ask in [GitHub Discussions](#). Happy coding ! 🚀